
Book Recommendation System

Sarah Borsotto

Halicioğlu Data Science Institute
University of California San Diego
sborsott@ucsd.edu

Ryan Clement

Halicioğlu Data Science Institute
University of California San Diego
rrclement@ucsd.edu

Felipe Lorenzi

Halicioğlu Data Science Institute
University of California San Diego
florenzi@ucsd.edu

1 Introduction

1.1 Problem Statement

The rise of digital platforms has changed the way consumers search for and purchase books, shifting business from traditional bookstores to online marketplaces. E-commerce offers vast selections, convenience, and personalized experiences to readers, establishing itself as the leading market Pleskach et al. [2024]. While this new approach has increased product accessibility, it has also introduced new challenges for readers. The overwhelming number of book options available online makes it difficult for users to find their next good read. According to Chen, Shang, and Kao (2009), excessive information in an online shopping environment can lead to decision fatigue, reducing consumer confidence and making it harder for users to finalize purchases Chen et al. [2009]. Consumers who face an overwhelming number of options are more likely to defer purchasing decisions or resort to familiar choices rather than explore new content Scheibehenne et al. [2010]. This behavior is detrimental not only to readers, who miss out on books that align with their interests, but also to publishers, authors, and booksellers, whose success depends on sales.

To address this issue, various recommendation systems have been implemented to help users navigate book selections. Platforms such as Goodreads offer recommendation features based on a user's reading history, preferred genres, and community reviews using a collaborative filtering approach. However, these systems primarily suggest books based on direct user engagement rather than deeper content-based similarities. They also fail to account for books that users may not have enjoyed, potentially leading to suboptimal recommendations Mhammedi et al. [2024]. Therefore, a more advanced recommendation system that not only analyzes explicit user preferences but also incorporates book content and attribute filters may enhance personalization. This improvement can lead to higher customer engagement by providing readers with more relevant and appealing book suggestions, which ultimately drives increased sales for authors, publishers, and booksellers.

1.2 Background

Two of the most widely used approaches in recommendation systems are Collaborative Filtering (CF) and Content-Based Filtering (CBF). Collaborative filtering identifies patterns in user behavior by analyzing past interactions. It works by finding users with similar reading histories and recommending books that these similar users have enjoyed. This method is effective in surfacing popular and well-received books but struggles with challenges such as sparse data matrices and the cold start problem, where new books and users lack enough data to generate reliable recommendations Roy and Dutta [2022]. Furthermore, CF does not explicitly take into account a user's individual preferences beyond what is inferred from similar users.

On the other hand, content-based filtering recommends books by analyzing their content, such as genre, themes, and writing style, and matching them to a user’s explicitly stated preferences. Although this method personalizes recommendations based on individual tastes, it lacks novelty and diversity; users are often recommended books that are too similar to what they have already read. Moreover, CBF requires extensive metadata, which may not always be available, making it difficult to generate high-quality recommendations in some cases Roy and Dutta [2022].

To address these limitations, a hybrid recommendation model is often used to combine the strengths of both CF and CBF. This approach integrates user behavior patterns for diversity while incorporating content similarity for relevance Wayesa et al. [2023]. Our proposed solution follows this hybrid methodology, utilizing collaborative filtering to recommend books based on user interactions and ratings and content-based filtering to analyze semantic embeddings for books with similar descriptions. Additionally, we introduce user-selected filters such as page count and publication date to further refine recommendations. We hope to address the shortcomings of individual recommendation techniques through a weighted and filtered hybrid recommendation approach that promotes relevant and novel recommendations, leading to improved user satisfaction levels.

2 Data

For our problem, we need a comprehensive dataset that includes user interactions, explicit ratings, book details (titles, authors, and editions), book descriptions, and book metadata (genres, publication year, number of pages, and other attributes). This data is essential for building a recommendation system that effectively combines collaborative filtering (interactions and ratings) and content-based filtering (descriptions and metadata) to provide accurate book suggestions.

2.1 Data Collection

Our dataset was obtained from the UCSD Book Graph, a publicly available collection of book-related data sourced from Goodreads in late 2017. The dataset was compiled by scraping users’ public bookshelves, ensuring that only publicly visible information was included, with anonymized user and review IDs to maintain privacy McAuley [2017].

The dataset consists of three main components:

Book Metadata – Contains detailed information on books, including titles, authors, publication details, formats, url, and page count, as well as popularity metrics like average ratings and text review counts. Books are also linked to author data based on author ID.

User-Book Interactions – Represents user engagement with books, include timestamps, ratings (1-5), read status, and review IDs, providing a comprehensive view of reading behaviors.

Given the large volume of data, over 228 million interactions, we focused on a specific genre for our study: Comics and Graphic Novels. This includes 89,411 books and 7,347,630 user interactions.

2.2 Data Preprocessing

We preprocessed the dataset by cleaning and structuring the data for both books and user interactions. Since many books exist in multiple editions and formats, we grouped different versions of the same book under a single work id to standardize our representation. The best book for each work_id was determined by selecting the edition with the highest rating count within each work_id group. Additionally, work_ids were stored alongside their corresponding book_ids for future mapping in the graph database. Book_ids in the user data were converted to their respective work_ids, with the best book_id for each work_id used as the final reference. This approach allowed us to treat all editions as the same book, preventing redundancy in recommendations. Given that our semantic analysis relies on book descriptions, we aimed to avoid cases where multiple versions of the same book would be returned as separate recommendations.

To further improve the quality of our data, we applied the following filtering steps:

1. Excluded non-English books to ensure better vector embeddings, as multilingual descriptions could introduce inconsistencies in our content-based filtering.

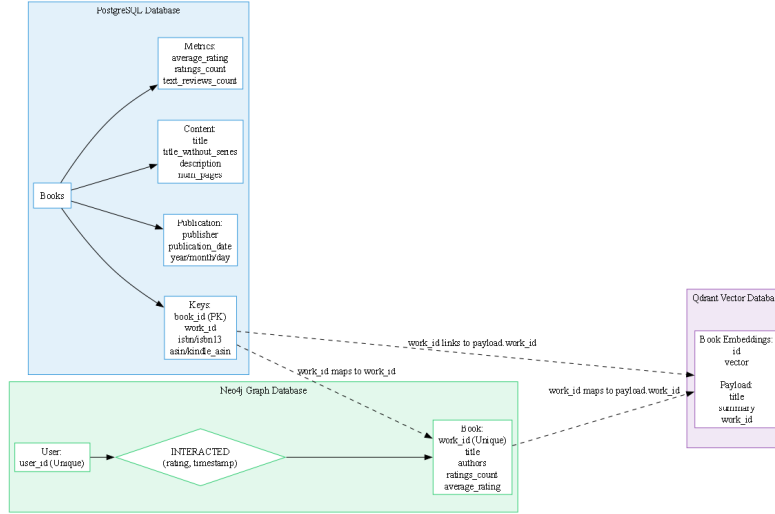


Figure 1: Data Pipeline

2. Removed books with missing descriptions, as descriptions were crucial for generating semantic embeddings.
3. Imputed missing data by selecting the best available description, publication year, and number of pages for each work id based on the most rated book in each work id group.

For the semantic search, we created vector embeddings for book descriptions using the all-MiniLM-L6-v2 transformer model, which provides a numerical representation of each book’s content that we then used to compute book similarity based on textual semantics.

After preprocessing, we retained the following features: **work_id** (unique identifier for a book across editions), **title**, **author_ids**, **description** (processed into vector embeddings), **publication date** (extracted from publication year, month, and day), **ratings count**, **average rating**, **number of pages**, **format**, **publisher**, and **link**. Subsets of the data were used for the graph and semantic analysis, namely **work_id**, **title**, and **description**.

The user interactions dataset contained logs of user engagement with books, including viewing, rating, and reading activity. To align with our work-based book representation, we added **work_id** to each interaction utilizing the mapping between **book_id** and **work_id**.

Since our model needed user feedback to learn preferences, we retained only the most relevant interaction features: **user_id** (anonymized), **work_id** (for linking interactions to books), **rating** (explicit user feedback as 1-5 and implicit interactions as 0), and **timestamp**.

Structuring our dataset this way ensured that our recommendation system could identify books based on meaningful content similarity and consolidated user interactions while avoiding redundant recommendations caused by multiple editions of the same book. An overview of the data pipeline can be found in Figure 1.

3 Databases

For this project, we utilized various databases and applications to build an efficient book recommendation system. Neo4j was used for graph representation of user-book interactions, allowing us to model relationships between users, books, and ratings for collaborative filtering. Qdrant, a vector database, was employed to store book descriptions as embedding vectors, enabling detailed and efficient similarity searches for content-based recommendations. PostgreSQL was used to manage detailed book metadata, such as author information, publication details, and formats, for structured and reliable data retrieval.

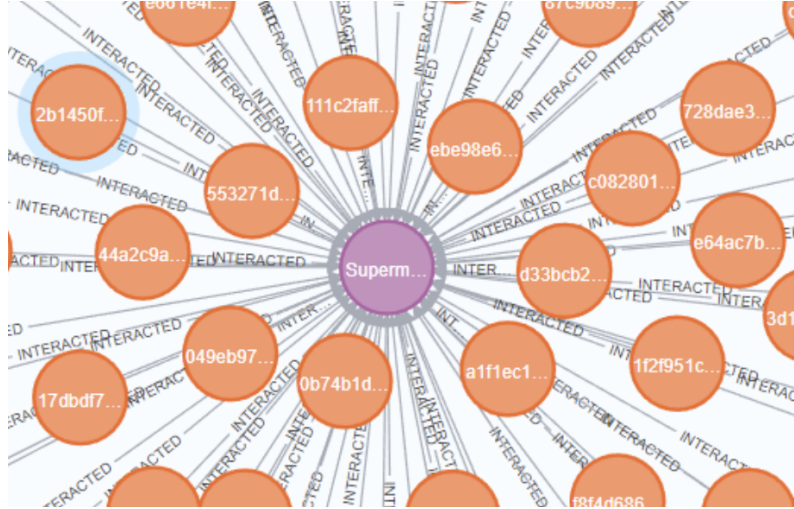


Figure 2: Neo4J Graph Example - Shows a book node and user interactions with that book

We used Streamlit to implement the recommender system logic and user interface, providing an interactive and user-friendly platform for users to explore book recommendations. Finally, Docker was used to containerize the application and databases, enabling seamless integration of multiple applications within a single service. This combination of tools allowed us to build a hybrid recommendation system that integrates collaborative and content-based filtering with additional attribute filters in an easy-to-use manner.

3.0.1 Neo4J

Neo4j was used to model and store user-book interactions as a graph database. In our Neo4j schema, users and books were represented as nodes, with edges linking them through interactions that captured user engagement with books (Figure 2).

Each book node contained essential metadata, including title, authors, work_id, ratings count, and average rating, ensuring that relevant information was readily available for queries and analysis. User nodes were defined by a unique user_id, allowing us to track individual interactions. The relationships between users and books were represented as edges, storing key attributes such as ratings and timestamps, to represent user behavior. Due to the large volume of data, over 7 million entries, interactions were inserted in batches.

By leveraging Neo4j's graph-based structure, relationship-based queries and recommendation generation became feasible. We could identify similar users and find book recommendations through user engagement patterns.

3.0.2 QDrant

Qdrant was used to store book description embeddings for more efficient similarity searches. We processed book descriptions using the sentence transformer model 'all-MiniLM-L6-v2', which converted textual descriptions into high-dimensional vector embeddings. A vector embedding is a numerical representation of text, encoded in a high-dimensional space, that captures semantic meaning and relationships between documents, resulting in fast similarity comparisons. The all-MiniLM-L6-v2 model generates embeddings using a transformer-based neural network, where input text is processed through multiple self-attention layers, and token representations from the final hidden layer are aggregated to produce a fixed-size vector. # add a citation These embeddings captured semantic similarities between books, allowing us to recommend books with similar themes or summaries.

For each book's work_id, we generated a corresponding vector embedding and stored it in Qdrant (Figure 3). The work_id was included as a payload field alongside the embeddings, serving as

```

# Create batch of text to encode
texts = df[column_name].tolist()

# Encode all texts at once (more efficient)
vectors = model.encode(texts, show_progress_bar=True)

# Create point objects for Qdrant
book_vectors = []

for i, (index, row) in enumerate(df.iterrows()):
    book_vectors.append({
        "id": index, # Use dataframe index as ID
        "vector": vectors[i].tolist(),
        "payload": {
            "title": row.get("title", ""), # Store title in payload
            "summary": row.get(column_name, "") # Store summary text for reference
            # Add additional metadata fields here if needed
        }
    })

```

Figure 3: QDrant Vector Embeddings

```

CREATE TABLE books (
    book_id INTEGER PRIMARY KEY,
    work_id INTEGER,
    isbn VARCHAR(50),
    isbn13 VARCHAR(50),
    asin VARCHAR(50),
    kindle_asin VARCHAR(50),
    series INTEGER[],
    format VARCHAR(20),
    publisher VARCHAR(50),
    author_ids INTEGER[],
    link VARCHAR(100),
    publication_date DATE,
    publication_year INT,
    publication_month INT,
    publication_day INT,
    title VARCHAR(255),
    title_without_series VARCHAR(255),
    is_ebook BOOLEAN,
    average_rating FLOAT,
    text_reviews_count INTEGER,
    ratings_count INTEGER,
    description TEXT,
    num_pages INTEGER
);

CREATE TABLE authors (
    author_id INTEGER PRIMARY KEY,
    name VARCHAR(255),
    average_rating FLOAT,
    text_reviews_count INTEGER,
    ratings_count INTEGER
);

```

Figure 4: Postgres Schema

a primary key for efficient retrieval and mapping. This structure facilitated fast nearest-neighbor searches, making it possible to identify books with the most relevant content-based matches.

3.0.3 Postgres

PostgreSQL provided a reliable way to retrieve and display book information after recommendations were generated, making the presentation clear and visually appealing for users. It also provided filtering capabilities based on publication year, page count, and other attributes, further refining the final recommendation.

The books table contained essential metadata such as work_id, title, author_id, description, publication date, ratings count, average rating, number of pages, link, and publisher. Only the best book for each work id, most rated edition, was stored.

Additionally, the authors table stored metadata about authors, linking author IDs from the books table to their respective names, which was crucial for displaying author names alongside recommended books (Figure 4).

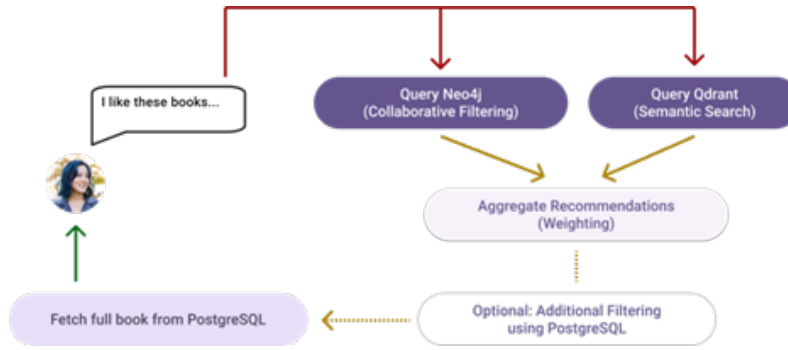


Figure 5: Enter Caption

4 Methodology

This section describes the overall architecture and functionality of the hybrid book recommendation system. The system integrates three main components: PostgreSQL for metadata storage, Neo4j for collaborative filtering, and Qdrant for semantic search. Streamlit is used to provide an interactive user interface that combines these components into a seamless recommendation workflow.

4.1 main.py (streamlit)

The `main.py` file serves as the entry point for the application. It uses Streamlit to create an interactive user interface and orchestrates the interactions between the PostgreSQL, Neo4j, and Qdrant connectors. The workflow is divided into three main steps:

1. **Search for a Book:** The user provides a description of the type of book they are looking for. Qdrant performs a semantic search to retrieve relevant books.
2. **Select Additional Books:** Based on the initial selection, Qdrant retrieves semantically similar books. The user can select additional books to refine the recommendations.
3. **Generate Recommendations:** The system combines collaborative filtering recommendations from Neo4j and semantic recommendations from Qdrant. If no personalized recommendations are found, popular books are retrieved as a fallback.

The final recommendations are enriched with metadata from PostgreSQL and displayed to the user.

4.2 Neo4J connector

The `neo4j_connector.py` file handles interactions with the Neo4j graph database. It provides methods for collaborative filtering and retrieving popular books. Key queries include:

4.2.1 Collaborative Filtering Recommendations

This query identifies books that are frequently interacted with by users who have similar preferences to the current user. Two variations of the query are used:

- For a single input book:

```

1 MATCH (user:User)-[r:INTERACTED]->(seed:Book)
2 WHERE seed.work_id IN $book_ids
3 MATCH (user)-[r2:INTERACTED]->(recommendation:Book)
4 WHERE NOT recommendation.work_id IN $book_ids
5 WITH recommendation, COUNT(DISTINCT user) as userOverlap, AVG(
6   r2.rating) as avgRating
7 RETURN recommendation.work_id as work_id, recommendation.title
   as title,
   recommendation.average_rating as average_rating,
   recommendation.ratings_count as ratings_count,

```

```

8         avgRating as user_rating , userOverlap as score
9     ORDER BY score DESC, avgRating DESC
10    LIMIT $limit
11

```

- For multiple input books:

```

1     MATCH (user:User)-[r:INTERACTED]->(seed:Book)
2     WHERE seed.work_id IN $book_ids AND r.rating >= 3
3     MATCH (user)-[r2:INTERACTED]->(recommendation:Book)
4     WHERE r2.rating >= 3 AND NOT recommendation.work_id IN
      $book_ids
5     WITH recommendation, COUNT(DISTINCT user) as userOverlap, AVG(
      r2.rating) as avgRating
6     RETURN recommendation.work_id as work_id, recommendation.title
      as title ,
7           recommendation.average_rating as average_rating ,
      recommendation.ratings_count as ratings_count ,
8           avgRating as user_rating , userOverlap as score
9     ORDER BY score DESC, avgRating DESC
10    LIMIT $limit
11

```

4.2.2 Popular Books

This query retrieves the most popular books based on their ratings count and average rating:

```

1 MATCH (book:Book)
2 WHERE book.ratings_count >= 1000
3 RETURN book.work_id as work_id, book.title as title ,
4        book.average_rating as average_rating, book.ratings_count as
      ratings_count ,
5        book.ratings_count * book.average_rating as score
6 ORDER BY score DESC
7 LIMIT $limit

```

4.3 Qdrant connector

The `qdrant_connector.py` file handles interactions with the Qdrant vector database. It provides methods for semantic search and retrieving book details. Key queries include:

4.3.1 Retrieve a Book by ID

This method retrieves a single book from the Qdrant collection based on its ID:

```

1 points = self.client.retrieve(
2     collection_name=self.collection_name ,
3     ids=[book_id]
4 )

```

4.3.2 Search for Similar Books

This method searches for books similar to a given query text or a specific book:

```

1 search_results = self.client.search(
2     collection_name=self.collection_name ,
3     query_vector=query_vector ,
4     limit=limit + (len(exclude_ids) if exclude_ids else 0) ,
5     score_threshold=0.0
6 )

```

The query vector is generated using a pre-trained sentence transformer model.

4.4 Postgres connector

The `postgres_connector.py` file handles interactions with the PostgreSQL database. It provides methods for retrieving book metadata and descriptions. Key queries include:

4.4.1 Retrieve Book Metadata

This query retrieves enriched metadata for a list of books based on their `work_id`:

```
1 SELECT b.title , b.isbn , b.num_pages , b.average_rating , b.publisher , b.  
   description ,  
2       b.link , b.publication_date ,  
3       (SELECT string_agg(a.name , ',' ) FROM authors a WHERE a.author_id  
   = ANY(b.author_ids)) AS author_names  
4 FROM books b  
5 WHERE b.work_id = ANY(%s)
```

4.4.2 Retrieve Description Mapping

This query retrieves a mapping of `work_id` to book descriptions:

```
1 SELECT work_id , description FROM books
```

4.5 Docker

Installation

Downloads

1. Clone the repository into a folder.
2. Download and open **Docker Desktop**.
3. (Optional) Download the **UCSD Goodreads Dataset (Comics & Graphic)**.

Cleaning Data

Since the clean data is too large to store on GitHub, you must clean it yourself. Run all cells in the following Jupyter notebooks:

- `data-processing/goodreads_data_pipeline.ipynb`
- `data-processing/interactions_work_id.ipynb`
- `data-processing/goodreads_data_pipeline_postgres.ipynb`

Moving Clean Data to App Container

After running the data cleaning notebooks, move the following files:

- `data-processing/data/goodreads_books_comics_graphic_cleaned_neo4j.csv`
`data-processing/data/goodreads_interactions_comics_graphic_cleaned.csv`
into
`BookRec/neo4j_import/`.
- `data-processing/data/goodreads_books_cleaned.csv`
`data-processing/data/goodreads_authors_cleaned.csv`
into
`BookRec/postgres_init/`.

Loading Data into Neo4j

Now that the data can be detected by the container, upload it to the Neo4j database.

Steps

1. Run the following command:

```
1 docker-compose up -d
2
```

2. Navigate to <http://localhost:7474/browser/> and log in with:

- Username: neo4j
- Password: dsc202friends

3. Run the following code block in the Neo4j terminal:

```
1 CREATE CONSTRAINT book_unique IF NOT EXISTS
2 FOR (b:Book) REQUIRE b.work_id IS UNIQUE;
3
4 CREATE CONSTRAINT user_unique IF NOT EXISTS
5 FOR (u:User) REQUIRE u.user_id IS UNIQUE;
6
7 LOAD CSV WITH HEADERS FROM 'file:///
8 goodreads_books_comics_graphic_cleaned_neo4j.csv' AS row
9 MERGE (b:Book {work_id: row.work_id})
10 SET b.title = row.title ,
11     b.authors = row.authors ,
12     b.ratings_count = toInteger(row.ratings_count) ,
13     b.average_rating = toFloat(row.average_rating);
14
15 :auto CALL {
16     LOAD CSV WITH HEADERS FROM 'file:///
17 goodreads_interactions_comics_graphic_cleaned.csv' AS row
18     MERGE (u:User {user_id: row.user_id})
19     WITH row, u
20     MATCH (b:Book {work_id: row.work_id})
21     MERGE (u)-[r:INTERACTED]->(b)
22     SET r.rating = toFloat(row.rating) ,
23         r.timestamp = row.timestamp
24     RETURN count(*) AS batchCount
25 } IN TRANSACTIONS OF 500 ROWS
RETURN "Completed" AS status;
```

Usage

1. Ensure that the Docker container is running using:

```
1 docker-compose up -d
2
```

If not, refer to the **Installation** section above.

2. Navigate to the application site at <http://localhost:8501/>.
3. Follow the on-screen instructions to get your book recommendations!

4.6 Results

Below is a series of screenshots displaying the homepage and recommendation features of the implemented BookRec app.

For an in-depth walkthrough, visit this video.

Key Highlights:

Upon entering the site, users are presented with two primary options: they can either rate books they have read or describe the type of book they are seeking. If users choose to rate books, they can search for titles and select those they have already read. Following this selection, they are prompted to

User Identification
 Enter your User ID (if new, leave blank):

 Additional Filters (Optional):

Recommendation Mode
 Choose how you want to get recommendations:
☒ Rate Books
☐ Describe Your Query

Rate Books
 Select up to 5 books and rate each (ratings from 1 to 5).
 Select books to rate:

Figure 6: Homepage

Choose how you want to get recommendations:
☒ Rate Books
☐ Describe Your Query

Rate Books
 Select up to 5 books and rate each (ratings from 1 to 5).
 Select books to rate:

Rate "Batman: Inform!":

 1.00 5.00

Rate "Superman Birthright":

 1.00 5.00

Rate "The Flash: Born to Run":

 1.00 5.00

Figure 7: User enters ratings for books

provide ratings for the chosen books. Once the ratings are submitted, users can generate personalized book recommendations based on their input.

Alternatively, if users prefer not to rate books, they can utilize the "Describe Your Query" button to articulate the specific type of book they wish to have recommended. This feature allows for a more tailored approach to finding suitable reading material.

Additionally, users have the option to refine their recommendations using the filter options available at the top of the page. This functionality enhances the user experience by allowing for more precise searches based on individual preferences.

Monster: Perfect Edition, Vol. 7
 ISBN: 1421589124
 Authors: Naoki Urasawa, Camilla Nish
 Pages: 410
 Average Rating: 4.57
 Publisher: VIC Media LLC
 Description: Johan is a cold and calculating killer with a mysterious past, and brilliant Dr. Kenzo Tenma is the only one who can stop him! Conspiracy and serial murder open the door to a compelling, intricately woven plot in this masterpiece of suspense. Although Eva fears an unfathomable hatred for Tenma, she is the only person who can testify as a witness to prove his innocence. She stubbornly refuses to do so until a discussion with Dr. Rochnstein brings back old memories of her time with Tenma. With those thoughts come memories of three gunshots, and that it was Johan who Eva saw that night! Meanwhile the evil hand of Robert inches steadily closer to Eva with the intent to kill... Tenma, Nina, and Johan all find themselves heading for the one place that holds the key to the mystery—the "Red Rose Mansion." What really happened there and why? Who is Franz Bonaparte? Will the secret of the birth of the Monster ever be revealed?
[More Info](#)

Vinland Saga, Omnibus 3
 ISBN: None
 Authors: Makoto Nakamura
 Pages: #12
 Average Rating: 4.51
 Publisher: None

Figure 8: Recommendations

Enter your User ID (if new, leave blank)

Additional Filters (Optional)

Oldest Publication Date
YYYY-MM-DD

Maximum Number of Pages
1000

☐ Available as Ebook

Format
No Preference

Minimum Average Rating
0.0

Minimum Rating Count
0

Recommendation Mode

Choose how you want to get recommendations:

☒ Rate Books

☐ Describe Your Query

Figure 9: Additional Filters for Search

5 Conclusion

5.1 Lessons Learned

In this project, we successfully developed a hybrid book recommendation system by integrating collaborative filtering (Neo4j-based graph model), content-based filtering (semantic embeddings stored in Qdrant), and user preference filters. We generated recommendations based on similar users' reading habits and book summary information, leading to more relevant and novel recommendations.

Throughout the development process, we also gained key insights into the challenges and optimizations needed for large-scale recommendation systems. Neo4j proved to be powerful for modeling user-book interactions but required significant processing time due to the large dataset, highlighting the importance of indexing and batch processing techniques when loading the data. Docker made it much easier to integrate different components, manage our databases, and smoothly deploy the application. It handled the outputs and queries from our three databases smoothly, making it a powerful and reliable tool for keeping everything connected and running smoothly. We found that Qdrant embeddings performed well in identifying relevant books based on simple descriptions, such as finding books about mermaids. However, for more complex and detailed queries — like requesting a sci-fi novel about mermaids from the 1900s — the results were less precise, often missing some key details. Additionally, we noticed that collaborative filtering did not always produce the most accurate recommendations. For example, rating Superman and Batman highly led to a recommendation for Star Wars, when a more expected result would have been another superhero-related book. This suggests potential limitations in our collaborative filtering queries, which may need refinement through additional data, improved filtering techniques, or more advanced query logic to enhance recommendation accuracy.

5.2 Future Work

Future improvements to the system will focus on expanding the dataset to include additional genres, such as young adult and history, to create a more robust, practical, and widely applicable recommendation system. Given the large volume of data, a cloud-based implementation could be a more scalable solution. The dataset also includes user-generated reviews linked to specific books, which could be utilized to enhance the recommendation quality. These reviews are written in long form text responses, so Qdrant embeddings would be required in order to derive meaningful semantic insights. For example, we could potentially extract sentiment by distinguishing positive and negative reviews to further refine our collaborative filtering model. In addition to these improvements, we are eager to assess the overall quality of recommendations. Future work would involve collecting subjective user feedback by either comparing recommendations to actual user history or conducting user trials where participants rate the system's suggestions. This would help validate the effectiveness of our recommendations and ensure that users find the service valuable.

6 Appendix

References

- Y. C. Chen, R. A. Shang, and C. Y. Kao. The effects of information overload on consumers' subjective state towards buying decision in the internet shopping environment. *Electronic Commerce Research and Applications*, 8(1):48–58, 2009. doi: 10.1016/j.elerap.2008.09.001. URL <https://doi.org/10.1016/j.elerap.2008.09.001>.
- Julian McAuley. Ucsd book graph dataset. Online, 2017. URL <https://cseweb.ucsd.edu/~jmcauley/datasets/goodreads.html>. Last accessed: March 2025.
- S. Mhammedi, H. El Massari, N. Gherabi, and M. Amnai. Enhancing book recommendations on goodreads: A data mining approach based on random forest classification. In M. Ben Ahmed, A.A. Boudhir, R. El Meouche, and İ.R. Karas, editors, *Innovations in Smart Cities Applications Volume 7*, volume 938 of *Lecture Notes in Networks and Systems*. Springer, Cham, 2024. doi: 10.1007/978-3-031-54376-0_36. URL https://doi.org/10.1007/978-3-031-54376-0_36.
- Valentyna Pleskach, Yaroslav Kryvolapov, Hlib Kryvolapov, and Olena Zelikovska. Investigating e-commerce systems for book sales: From theoretical foundations to software development. *CEUR Workshop Proceedings*, 3909, 2024. URL https://ceur-ws.org/Vol-3909/Paper_21.pdf. Last accessed: March 2025.
- D. Roy and M. Dutta. A systematic review and research perspective on recommender systems. *Journal of Big Data*, 9:59, 2022. doi: 10.1186/s40537-022-00592-5. URL <https://doi.org/10.1186/s40537-022-00592-5>.
- Benjamin Scheibehenne, Rainer Greifeneder, and Peter M. Todd. Can there ever be too many options? a meta-analytic review of choice overload. *Journal of Consumer Research*, 37(3):409–425, 2010. doi: 10.1086/651235. URL <https://academic.oup.com/jcr/article-abstract/37/3/409/1827647>.
- F. Wayesa, M. Leranso, G. Asefa, et al. Pattern-based hybrid book recommendation system using semantic relationships. *Scientific Reports*, 13:3693, 2023. doi: 10.1038/s41598-023-30987-0. URL <https://doi.org/10.1038/s41598-023-30987-0>.